# Rsamtools

***Kasper D. Hansen***



## Dependencies

This document has the following dependencies:

```
library(Rsamtools)
```

Use the following commands to install these packages in R.

```
source("http://www.bioconductor.org/biocLite.R")
biocLite(c("Rsamtools")
```

## Overview

The *Rsamtools* packages contains functionality for reading and examining aligned reads in the BAM format.

## Other Resources

- The vignettes from the Rsamtools webpage.

- For representing more complicated alignments (specifically spliced alignments from RNA-seq), see the *GenomicAlignments* package.

# Rsamtools

The Rsamtools package is an interface to the widely used `samtools/htslib` library. The main functionality of the package is support for reading BAM files.

## The BAM / SAM file format

The SAM format is a text based representation of alignments. The BAM format is a binary version of SAM which is smaller and much faster. In general, always work with BAM.

The format is quite complicated, which means the R representation is also a bit complicated. This complication happens because of the following features of the file format

- It may contain unaligned sequences.
- Each sequence may be aligned to multiple locations.
- It supports spliced (split) alignments.
- It may contain reads from multiple samples.

We will not attempt to fully understanding all the intricacies of this format.

A BAM file can be sorted in multiple ways. If it is sorted according to genomic location and if it is also "indexed" it is possible to retrieve all reads mapping to a genomic location, something which can be very useful. In *Rsamtools* this is done by the functions `sortBam()` and `indexBam()`.

# scanBam

How to read a BAM file goes conceptually like this

1. A pointer to the file is created by the `BamFile()` constructor.
2. (Optional) Parameters for which reads to report is constructed by `ScanBamParams()`.
3. The file is being read according to these parameters by `scanBam()`.

First we setup a `BamFile` object:

```
bamPath <- system.file("extdata", "ex1.bam", package="Rsamtools")
bamFile <- BamFile(bamPath)
bamFile
```

```
## class: BamFile
## path: /Library/Frameworks/R.framework/Versions/3.2/Resources/lib.../e
## index: /Library/Frameworks/R.framework/Versions/3.2/Resource.../ex1.b
## isOpen: FALSE
## yieldSize: NA
## obeyQname: FALSE
## asMates: FALSE
## qnamePrefixEnd: NA
## qnameSuffixStart: NA
```

Some high-level information can be accessed here, like

```
seqinfo(bamFile)
```

```
## Seqinfo object with 2 sequences from an unspecified genome:
##   seqnames seqlengths isCircular genome
##   seq1           1575         NA   <NA>
##   seq2           1584         NA   <NA>
```

(obviously, `seqinfo()` and `seqlevels()` etc. are supported as well).

Now we read all the reads in the file using `scanBam()`, ignoring the possibility of selecting reads using `ScanBamParams()` (we will return to this below).

```
aln <- scanBam(bamFile)
length(aln)
```

```
## [1] 1
```

```
class(aln)
```

```
## [1] "list"
```

We get back a list of length 1; this is because `scanBam()` can return output from multiple genomic regions, and here we have only one (everything). We therefore subset the output; this again gives us a list and we show the information from the first alignment

```
aln <- aln[[1]]
names(aln)
```

```
##  [1] "qname"  "flag"   "rname"  "strand" "pos"    "qwidth" "mapq"
##  [8] "cigar"  "mrnm"   "mpos"   "isize"  "seq"    "qual"
```

```
lapply(aln, function(xx) xx[1])
```

```
## $qname
## [1] "B7_591:4:96:693:509"
##
## $flag
## [1] 73
##
## $rname
## [1] seq1
## Levels: seq1 seq2
##
## $strand
## [1] +
## Levels: + - *
##
## $pos
## [1] 1
##
## $qwidth
## [1] 36
##
## $mapq
## [1] 99
##
## $cigar
## [1] "36M"
##
## $mrnm
## [1] <NA>
## Levels: seq1 seq2
##
## $mpos
## [1] NA
##
## $isize
## [1] NA
##
## $seq
##   A DNAStringSet instance of length 1
##     width seq
## [1]    36 CACTAGTGGCTCATTGTAAATGTGTGGTTTAACTCG
##
## $qual
##   A PhredQuality instance of length 1
##     width seq
## [1]    36 <<<<<<<<<<<<<<<;<<<<<<<<<5<<<<<;:<;7
```

Notice how the `scanBam()` function returns a basic R object, instead of an S4 class. Representing the alignments as S4 object is done by the *GenomicAlignments* package; this is especially useful for access to spliced alignments from RNA sequencing data.

The names of the `aln` list are basically the names used in the BAM specification. Here is a quick list of some important ones

- `qname`: The name of the read.
- `rname`: The name of the chromosome / sequence / contig it was aligned to.
- `strand`: The strand of the alignment.
- `pos`: The coordinate of the left-most part of the alignment.
- `qwidth`: The length of the read.
- `mapq`: The mapping quality of the alignment.
- `seq`: The actual sequence of the alignment.
- `qual`: The quality string of the alignment.
- `cigar`: The CIGAR string (below).
- `flag`: The flag (below).

## Reading in parts of the file

BAM files can be extremely big and it is there often necessary to read in parts of the file. You can do this in different ways

1. Read a set number of records (alignments).
2. Only read alignments satisfying certain criteria.
3. Only read alignments in certain genomic regions.

Let us start with the first of this. By specifying `yieldSize` when you use `BamFile()`, every invocation of `scanBam()` will only read `yieldSize` number of alignments. You can then invoke `scanBam()` again to get the next set of alignments; this requires you to `open()` the file first (otherwise you will keep read the same alignments).

```
yieldSize(bamFile) <- 1
open(bamFile)
scanBam(bamFile)[[1]]$seq
```

```
##   A DNAStringSet instance of length 1
##     width seq
## [1]    36 CACTAGTGGCTCATTGTAAATGTGTGGTTTAACTCG
```

```
scanBam(bamFile)[[1]]$seq
```

```
##   A DNAStringSet instance of length 1
##     width seq
## [1]    35 CTAGTGGCTCATTGTAAATGTGTGGTTTAACTCGT
```

```
## Cleanup
close(bamFile)
yieldSize(bamFile) <- NA
```

The other two ways of reading in parts of a BAM file is to use `ScanBamParams()`, specifically the `what` and `which` arguments.

```
gr <- GRanges(seqnames = "seq2",
              ranges = IRanges(start = c(100, 1000), end = c(1500,2000))
params <- ScanBamParam(which = gr, what = scanBamWhat())
aln <- scanBam(bamFile, param = params)
names(aln)
```

```
## [1] "seq2:100-1500"  "seq2:1000-2000"
```

```
head(aln[[1]]$pos)
```

```
## [1] 66 68 68 72 73 77
```

Notice how the `pos` is less than what is specified in the `which` argument; this is because the alignments overlap the `which` argument. The `what=scanBamWhat()` tells the function to read everything. Often, you may not be interested in the actual sequence of the read or its quality scores. These takes up a lot of space so you may consider disabling reading this information.

## The CIGAR string

The “CIGAR” is how the BAM format represents spliced alignments. For example, the format stored the left most coordinate of the alignment. To get to the right coordinate, you have to parse the CIGAR string. In this example “36M” means that it has been aligned with no insertions or deletions. If you need to work with spliced alignments or alignments containing insertions or deletions, you should use the *GenomicAlignments* package. #### Flag

An alignment may have a number of “flags” set or unset. These flags provide information about the alignment. The flag integer is a representation of multiple flags simultanously. An example of a flag is indicating (for a paired end alignment) whether both pairs have been properly aligned. For more information, see the BAM specification.

In *Rsamtools* there is a number of helper functions dealing with only reading certain flags; use these.

## BAM summary

Sometimes you want a quick summary of the alignments in a BAM file:

```
quickBamFlagSummary(bamFile)
```

```
##                                    group |    nb of |    nb of | mean /
##                                       of |  records |   unique | records
##                                  records | in group |   QNAMEs | unique
## All records.......................... A |     3307 |     1699 | 1.95 /
##   o template has single segment.... S |        0 |        0 |   NA /
##   o template has multiple segments. M |     3307 |     1699 | 1.95 /
##       - first segment.............. F |     1654 |     1654 |    1 /
##       - last segment............... L |     1653 |     1653 |    1 /
##       - other segment.............. O |        0 |        0 |   NA /
##
## Note that (S, M) is a partitioning of A, and (F, L, O) is a partition
## Indentation reflects this.
##
## Details for group M:
##   o record is mapped.............. M1 |     3271 |     1699 | 1.93 /
##       - primary alignment......... M2 |     3271 |     1699 | 1.93 /
##       - secondary alignment....... M3 |        0 |        0 |   NA /
##   o record is unmapped............ M4 |       36 |       36 |    1 /
##
## Details for group F:
##   o record is mapped.............. F1 |     1641 |     1641 |    1 /
##       - primary alignment......... F2 |     1641 |     1641 |    1 /
##       - secondary alignment....... F3 |        0 |        0 |   NA /
##   o record is unmapped............ F4 |       13 |       13 |    1 /
##
## Details for group L:
##   o record is mapped.............. L1 |     1630 |     1630 |    1 /
##       - primary alignment......... L2 |     1630 |     1630 |    1 /
##       - secondary alignment....... L3 |        0 |        0 |   NA /
##   o record is unmapped............ L4 |       23 |       23 |    1 /
```

# Other functionality from Rsamtools

## BamViews

Instead of reading a single file, it is possible to construct something called a `BamViews`, a link to multiple files. In many ways, it has the same `Views` functionality as other views. A quick example should suffice, first we read everything;

```
bamView <- BamViews(bamPath)
aln <- scanBam(bamView)
names(aln)
```

```
## [1] "ex1.bam"
```

This gives us an extra list level on the return object; first level is files, second level is ranges.

We can also set `bamRanges()` on the `BamViews` to specify that only certain ranges are read; this is similar to setting a `which` argument to `ScanBamParams()`.

```
bamRanges(bamView) <- gr
aln <- scanBam(bamView)
names(aln)
```

```
## [1] "ex1.bam"
```

```
names(aln[[1]])
```

```
## [1] "seq2:100-1500"  "seq2:1000-2000"
```

### countBam

Sometimes, all you want to do is count… use `countBam()` instead of `scanBam()`.

# SessionInfo

```
## R version 3.2.1 (2015-06-18)
## Platform: x86_64-apple-darwin13.4.0 (64-bit)
## Running under: OS X 10.10.5 (Yosemite)
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] parallel  stats4    methods   stats     graphics  grDevices utils
## [8] datasets  base
##
## other attached packages:
##  [1] Rsamtools_1.20.4     Biostrings_2.36.4    XVector_0.8.0
##  [4] GenomicRanges_1.20.6 GenomeInfoDb_1.4.2   IRanges_2.2.7
##  [7] S4Vectors_0.6.5      BiocGenerics_0.14.0  BiocStyle_1.6.0
## [10] rmarkdown_0.8
##
## loaded via a namespace (and not attached):
##  [1] knitr_1.11              magrittr_1.5
##  [3] zlibbioc_1.14.0         GenomicAlignments_1.4.1
##  [5] BiocParallel_1.2.20     lattice_0.20-33
##  [7] hwriter_1.3.2           stringr_1.0.0
##  [9] tools_3.2.1             grid_3.2.1
## [11] Biobase_2.28.0          latticeExtra_0.6-26
## [13] lambda.r_1.1.7          futile.logger_1.4.1
## [15] htmltools_0.2.6         yaml_2.1.13
## [17] digest_0.6.8            RColorBrewer_1.1-2
## [19] formatR_1.2             futile.options_1.0.0
## [21] bitops_1.0-6            evaluate_0.7.2
## [23] stringi_0.5-5           ShortRead_1.26.0
```